ENGINEERING HANDBOOK · DETAILED EDITION

Production RAG Architectures

Ingestion se deployment tak — chunking, embeddings, vector indexes, hybrid search, agentic loops, observability, cost aur security. Har concept diagram, worked example, sahi (corrected) code aur interview angle ke saath.

LangChain / LangGraph | ChromaDB · PGVector | BM25 + RRF | HNSW | FastAPI · Docker

Prepared for Vaibhav · November 2026 · Built from your short notes, expanded and fact-checked

Contents

Padhne ka tareeka: pehle Ch 1–3 (foundation), phir Ch 5 (failure modes — sabse zyada interview mein poocha jaata hai), phir baaki.

1. RAG Fundamentals
Why RAG, two-phase pipeline, the math, grounding, citations, LCEL chain

2. Setup & Ingestion
uv, dependencies, document loaders, metadata design

3. Chunking
4 variables, 4 strategies, parent-child, compression, how to tune

4. Embeddings & Vector Stores
Embedding vs chat models, dimensions, distance metrics with numbers, Chroma

5. The 5 Production Failure Modes
Bad chunking, embedding mismatch, noise, overflow, hallucination; BM25, RRF, reranking, token budgets

6. Advanced Architectures
Contextual Retrieval, Agentic RAG (LangGraph), Graph RAG, Multimodal / ColPali

7. Observability & Evaluation
Traces, metrics, JSON logs, LangSmith, RAG eval metrics

8. Index Tuning & Cost
HNSW internals, M / ef, pgvector, cost tiers, quantization math

9. FastAPI, Security & Deployment
Request pipeline, hardened main.py, Docker, Render

10. Interview Q&A
15 questions with model answers

Appendix A
Corrections to the original notes

Appendix B
One-page cheat sheet

IS DOCUMENT MEIN BOXES KA MATLAB

■ Seedhi Baat — simple Hinglish intuition · ■ Note — extra detail · ■ Watch out — common galti · ■ Correction — original notes mein jo galat/outdated tha · ■ Interview — ye poocha jaata hai

01 RAG Fundamentals

Retrieval-Augmented Generation: LLM ko jawab dene se pehle tumhare documents se relevant text "pakda" dena.

1.1 Problem kya hai jo RAG solve karta hai?

An LLM only knows what was in its training data, frozen at a cutoff date. It knows nothing about your company's internal docs, last week's policy change, or a PDF a user just uploaded. When asked anyway, it does what it was trained to do — produce fluent, likely-sounding text — which is exactly how **hallucinations** happen.

RAG fixes this without retraining the model: at question time, **search** your own knowledge base, **paste** the most relevant passages into the prompt, and instruct the model to answer **only from those passages**. The model becomes a reader-and-summarizer of evidence instead of a memory.

SEEDHI BAAT

LLM = bahut smart student jisne exam se pehle tumhari company ki book kabhi nahi padhi. RAG = open-book exam. Question aata hai → tum book mein se sahi 3–5 paragraph nikal ke student ko de dete ho → student unhi se answer likhta hai, aur bata deta hai kaunse page se liya.

RAG vs Fine-tuning vs Long Context

Approach	What it changes	Good for	Weak at
RAG	What the model *sees* at query time	Fresh / private / large knowledge, citations, access control per user	Depends entirely on retrieval quality
Fine-tuning	Model weights	Style, format, tone, domain jargon, task behaviour	Adding facts reliably; updating knowledge needs retraining; no citations
Long context (paste everything)	Prompt size	Small corpora (a few docs), one-off analysis	Cost & latency per query, "lost in the middle", can't scale to 100K docs

In practice they combine: RAG for knowledge, fine-tuning (optional) for behaviour, long context to fit more retrieved chunks.

1.2 The Two-Phase Pipeline

Every RAG system has an **offline indexing phase** (runs when documents change) and an **online query phase** (runs per user request). Your notes' ASCII diagram mixes both; here they are separated, which is how you should explain it in an interview.

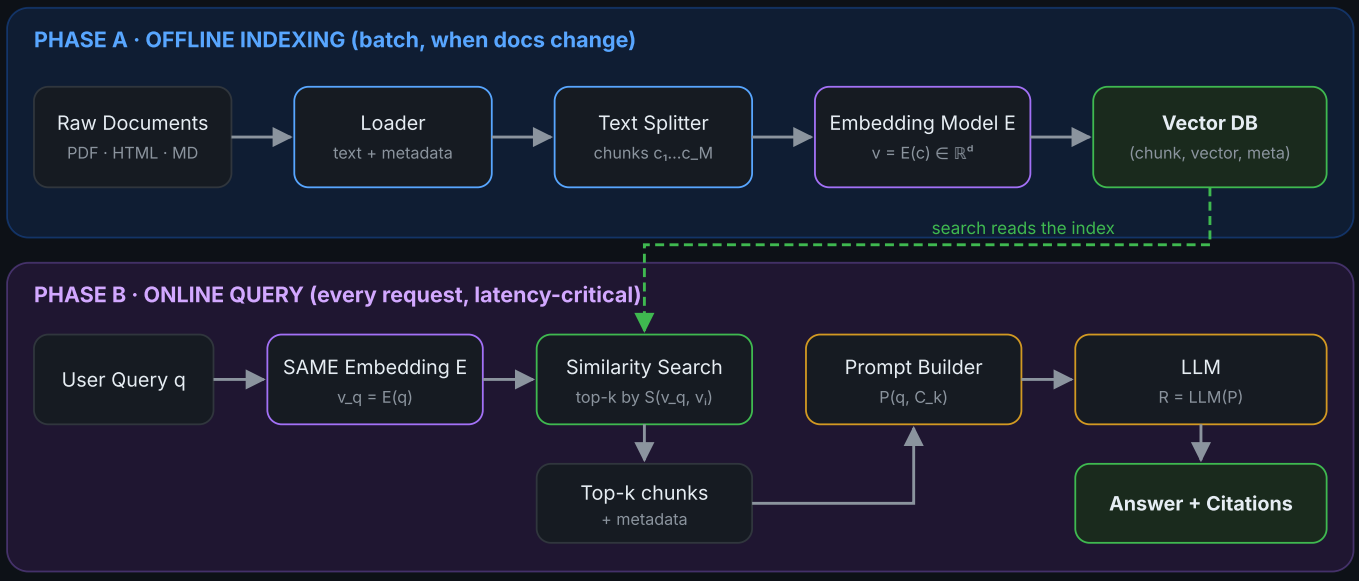


Fig 1.1 — Indexing happens once; query path runs per request. The embedding model MUST be identical in both phases; the original question q also goes into the prompt.

1.3 The Math, Step by Step

Step 1 — Indexing

> Documents $D = \{d_1, \ldots, d_N\}$ → split → chunks $C = \{c_1, \ldots, c_M\}$ $(M \gg N)$
> For every chunk: $\mathbf{v}_i = E(c_i) \in \mathbb{R}^d$ (e.g. $d = 1536$)
> Store tuple $(c_i, \mathbf{v}_i, m_i)$ where m_i = metadata {source, page, section, …}

Step 2 — Retrieval

> $\mathbf{v}_q = E(q)$ ← same E, otherwise the vectors live in different spaces and distances are meaningless
> C_k = top-k of $\{c_i\}$ ranked by $S(\mathbf{v}_q, \mathbf{v}_i)$, S = cosine similarity (usually)

Step 3 — Augmented generation

> $R = LLM(P(q, C_k))$ where P is a template: instructions + formatted chunks + question

WORKED MINI-EXAMPLE (D = 3, NUMBERS MADE UP)

Query "how to reset password" → $\mathbf{v}_q$ = [0.9, 0.1, 0.4]
Chunk A "Password reset steps…" → [0.8, 0.2, 0.5] · Chunk B "Quarterly revenue…" → [0.1, 0.9, 0.1]
cos(q, A) = (0.72+0.02+0.20) / (|q|·|A|) = 0.94 / (0.990 × 0.964) ≈ **0.985**
cos(q, B) = (0.09+0.09+0.04) / (0.990 × 0.911) ≈ **0.244** → A is retrieved, B is not.

INTERVIEW

"Why can't I switch embedding models without re-indexing?" — Because every model defines its own vector space. A query vector from model X compared with document vectors from model Y gives random similarity scores. Changing E ⇒ re-embed the full corpus (plan for this; store raw chunk text so you can).

1.4 Grounding: the Strict Prompt

Retrieval gives the model evidence; the prompt must make the model **obey** the evidence. Three ingredients: (1) answer *only* from context, (2) an explicit refusal sentence when context is insufficient, (3) forbid outside knowledge. The fixed refusal string is also useful operationally — you can count how often it appears to measure "retrieval misses" in production.

PYTHON

```python
from langchain_core.prompts import ChatPromptTemplate

rag_prompt = ChatPromptTemplate.from_messages([
    ("system",
     "You are a documentation assistant. Answer ONLY from the context below.\n"
     "If the context does not contain the answer, reply exactly:\n"
     "\"I don't have enough information to answer this question.\"\n"
     "Do not use outside knowledge. Cite sources as [source, page] after each claim.\n\n"
     "Context:\n{context}"),
    ("human", "{question}"),
])
```

WHY SYSTEM + HUMAN MESSAGES?

Your notes use `from_template` (a single human message). With chat models it is better to put rules in the **system** message and the user's question in the **human** message — models weight system instructions more strongly, and it reduces the chance that text inside retrieved documents ("ignore previous instructions...") gets treated as an instruction.

1.5 Source Citations

Each chunk carries metadata from the loader. Format it into the context block so the model can cite, and so the UI can show clickable sources.

PYTHON

```python
def format_docs_with_sources(docs) -> str:
    blocks = []
    for i, doc in enumerate(docs, start=1):
        src  = doc.metadata.get("source", "Unknown Source")
        page = doc.metadata.get("page", "N/A")
        blocks.append(f"[{i}] Source: {src} (page {page})\n{doc.page_content}")
    return "\n\n".join(blocks)
```

1.6 The LCEL Chain — what each piece does

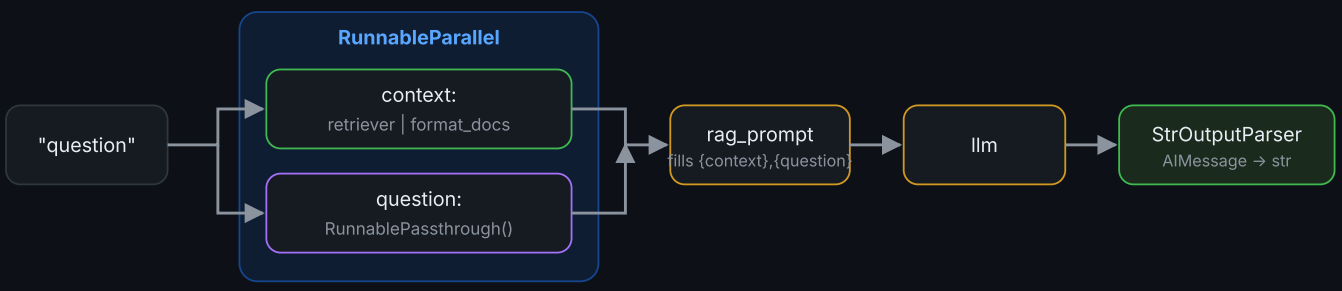


Fig 1.2 — The same input string fans out: one branch retrieves, the other passes the question through untouched.

PYTHON

```python
from langchain_core.runnables import RunnableParallel, RunnablePassthrough
from langchain_core.output_parsers import StrOutputParser
```

```python
retriever = vector_store.as_retriever(search_kwargs={"k": 4})

rag_chain = (
    RunnableParallel(
        context=retriever | format_docs_with_sources,   # str -> docs -> formatted str
        question=RunnablePassthrough(),                  # str unchanged
    )
    | rag_prompt
    | llm
    | StrOutputParser()
)

answer = rag_chain.invoke("How do I configure connection pooling?")
```

SEEDHI BAAT

`|` ka matlab Unix pipe jaisa: left ka output right ka input. `RunnableParallel` ek hi input ko do raaste pe bhejta hai aur result ek dict `{"context": ..., "question": ...}` banata hai — exactly wahi keys jo prompt template maangta hai.

WATCH OUT

This chain returns only a string, so the UI can't show the retrieved docs. In production, return both: use `RunnableParallel(...).assign(answer=...)` so the response carries `{"answer", "context_docs"}` — you need the docs for citations, logging and evaluation (Ch 7).

02 Environment Setup & Ingestion

Garbage in, garbage out — ingestion quality decides retrieval quality.

2.1 Project setup with uv

`uv` is a Rust-based Python package manager — much faster than pip, and it writes a lockfile (`uv.lock`) so every machine and the Docker image get identical versions.

```bash
uv init production-rag && cd production-rag
uv venv && source .venv/bin/activate

uv add langchain langchain-core langchain-community langgraph \
       langchain-openai langchain-anthropic langchain-chroma langchain-text-splitters langchain-
classic \
       python-dotenv pypdf pymupdf chromadb rank_bm25 tiktoken \
       fastapi uvicorn slowapi pydantic-settings

# Docker / Render ke liye requirements.txt chahiye:
uv export --format requirements-txt > requirements.txt
```

CORRECTION

The package is `fastapi`, not `fast-api` (that name would fail or install the wrong thing). Also added: `langchain-chroma` (the maintained Chroma integration), `langchain-text-splitters` and `tiktoken` (real token counting, used in Ch 5).

2.2 What a Loader actually produces

Every loader returns a list of `Document` objects: `page_content` (the text) + `metadata` (a dict). A 40-page PDF through `PyPDFLoader` gives 40 Documents, one per page, each with `{"source": "...pdf", "page": 7}`. Metadata is what later enables citations, filtering ("only HR docs"), access control and deletion when a file changes.

Loader	Speed	Layout / Tables	Metadata	Use when
PyPDFLoader	Fast	Plain text only	page, source	Simple text-heavy PDFs
PyMuPDFLoader	Fastest	Good text order, basic	page, fonts, rich file info	High-volume batch ingestion
UnstructuredPDFLoader	Slow	Best — tables, titles, columns	element type (Title, Table, NarrativeText)	Financial reports, multi-column papers
DirectoryLoader	Depends	Delegates	Inherited	Folder of files via glob
WebBaseLoader	Fast	HTML → text	URL, title	Docs portals, articles

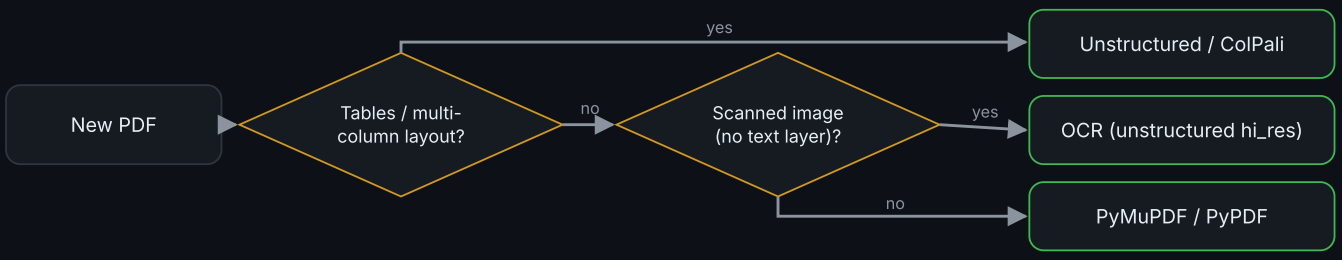


Fig 2.1 — Picking a PDF loader.

PYTHON

```python
from langchain_community.document_loaders import PyMuPDFLoader, DirectoryLoader

docs = DirectoryLoader(
    "docs/", glob="**/*.pdf", loader_cls=PyMuPDFLoader,
    show_progress=True, use_multithreading=True,
).load()

# Enrich metadata at ingestion time — you cannot filter on what you didn't store
for d in docs:
    d.metadata["doc_type"] = "spec" if "spec" in d.metadata["source"] else "general"
    d.metadata["ingested_at"] = "2026-11-09"
```

WATCH OUT — PRODUCTION INGESTION

- **Idempotency:** give every chunk a deterministic ID (e.g. `sha256(source + page + chunk_index)`). Re-running ingestion then *updates* instead of creating duplicates.
- **Updates/deletes:** when a file changes, delete all chunks where `source == file`, then re-insert. Without stored `source` metadata you can't do this.
- **Cleaning:** strip repeated headers/footers, page numbers and hyphenation (`config-\nuration`) before chunking — they pollute every embedding.

03 Chunking Architecture

Chunking minor step nahi hai — ye retrieval quality ki reedh ki haddi (backbone) hai. Galat boundary = search shuru hone se pehle hi meaning khatam.

3.1 Why chunk at all?

Two reasons. **(1) Embedding precision:** one vector must summarize the whole text it represents. A 30-page document squeezed into one vector becomes a blurry "average" of every topic in it and matches nothing sharply. **(2) Prompt budget:** you want to send the LLM the 3–5 most relevant passages, not 30 pages. So: split into pieces small enough to be about *one thing*, but large enough to be *self-contained*.

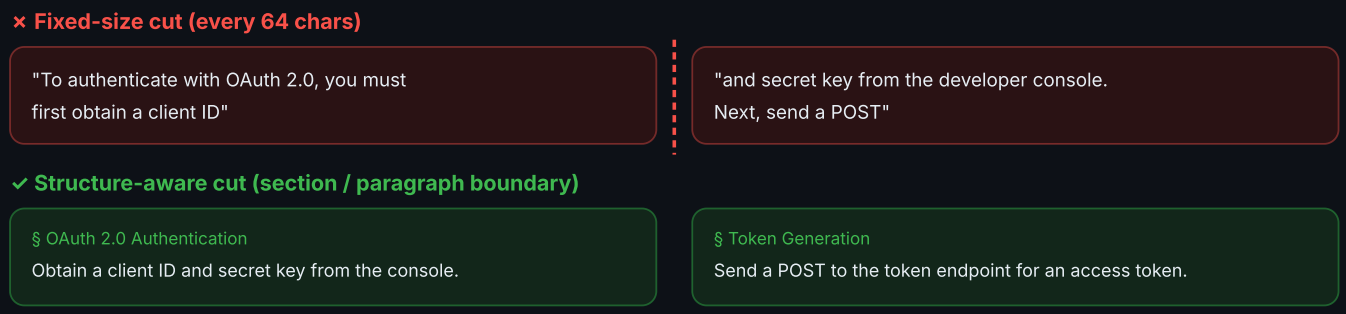


Fig 3.1 — Query "where do I get the secret key?" Chunk 1 of the bad split mentions OAuth but not the secret; chunk 2 mentions the secret but not OAuth. Neither matches well.

3.2 The Four Chunking Variables

Variable	What it is	Too small / low	Too large / high	Typical start
Chunk size	Length of each chunk (chars or tokens)	Fragments lose context ("it increased by 15%" — what did?)	Vector diluted by many topics; wastes prompt tokens	200–1,000 tokens; 500 chars ≈ 100–125 tokens
Overlap	Text repeated at the boundary of neighbours	Facts split across boundary get lost	Duplicate retrievals, bigger index	10–20% of size
Separators	Ordered list of split points	Order matters: try the biggest semantic unit first, fall back to smaller		`["\n\n","\n",". "," ",""]`
Content type	Format-specific rules	Code: split on `class` / `def`. Markdown: on headers. Tables: keep whole.		`from_language()`, MarkdownHeaderTextSplitter

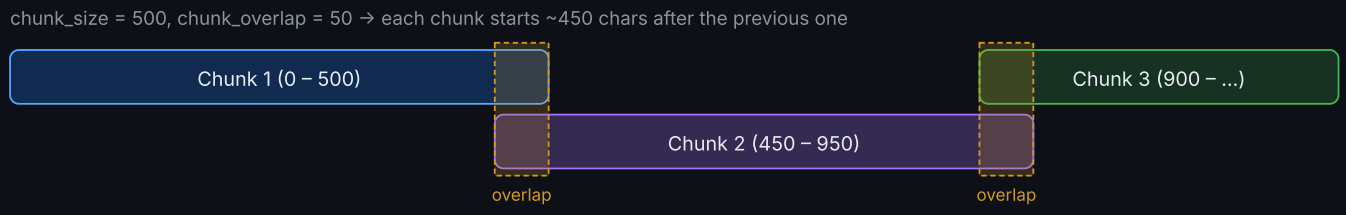


Fig 3.2 — Overlap is "cheap insurance": a sentence sitting on a boundary appears whole in at least one chunk.

WATCH OUT — CHARS VS TOKENS

`RecursiveCharacterTextSplitter(chunk_size=500)` counts **characters**, not tokens. English averages ~4 chars/token, so 500 chars ≈ 125 tokens — on the small side. To size in tokens use `RecursiveCharacterTextSplitter.from_tiktoken_encoder(chunk_size=400, chunk_overlap=60)`.

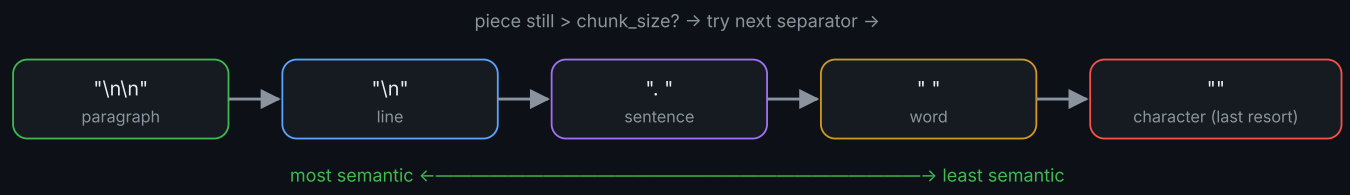
3.3 The Four Strategies

Strategy 1 — Fixed-size

Cut every N characters, ignore structure. Trivial and predictable, but slices words and sentences (Fig 3.1). **Verdict:** only as a baseline to compare against; don't ship it.

Strategy 2 — Recursive character (the default)

The splitter tries the first separator (`\n\n` = paragraph). Any piece still larger than `chunk_size` is split again with the next separator (`\n`), then sentences, then words, then raw characters as the last resort. Then small adjacent pieces are **merged back** up to chunk_size. Result: chunks break at the most meaningful boundary available.




```python
from langchain_text_splitters import RecursiveCharacterTextSplitter, Language

# Token-based sizing (recommended)
splitter = RecursiveCharacterTextSplitter.from_tiktoken_encoder(
    encoding_name="cl100k_base", chunk_size=400, chunk_overlap=60,
)
chunks = splitter.split_documents(docs)        # metadata is copied to every chunk

# Code-aware splitting: separators become class / def / blank lines
py_splitter = RecursiveCharacterTextSplitter.from_language(
    language=Language.PYTHON, chunk_size=1200, chunk_overlap=100,
)
```

PYTHON

Strategy 3 — Semantic chunking

Split text into sentences, embed each, and measure the **cosine distance between consecutive sentences**. Where the topic changes, distance spikes. With `breakpoint_threshold_type="percentile", amount=90`, every gap whose distance is in the top 10% becomes a chunk boundary.

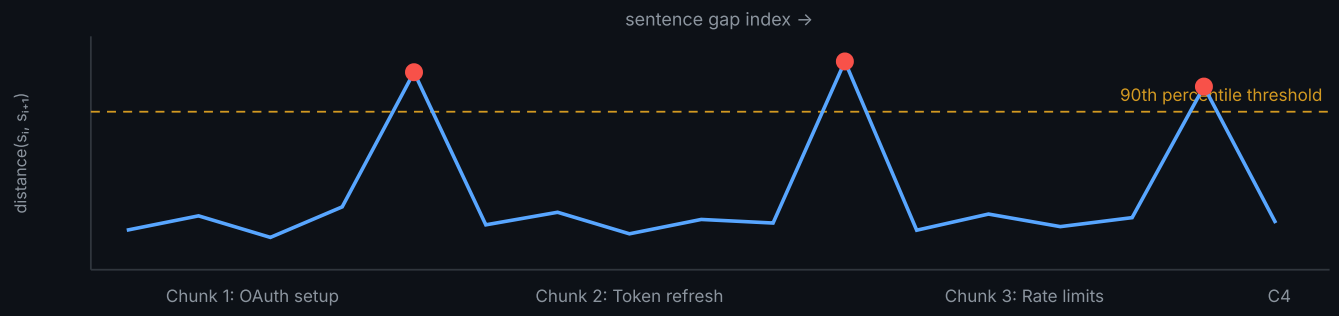


Fig 3.3 — Red spikes = topic shifts = chunk boundaries.

```python
from langchain_experimental.text_splitter import SemanticChunker
from langchain_openai import OpenAIEmbeddings

semantic_splitter = SemanticChunker(
    OpenAIEmbeddings(model="text-embedding-3-small"),
    breakpoint_threshold_type="percentile",   # also: "standard_deviation", "interquartile",
"gradient"
    breakpoint_threshold_amount=90,
)
semantic_chunks = semantic_splitter.split_documents(docs)
```

TRADE-OFF

Semantic chunking embeds *every sentence* at ingestion (extra cost + time), and chunk sizes become unpredictable (one giant chunk if a document never changes topic). Benchmarks show it is not consistently better than a well-tuned recursive splitter — measure on your own data before adopting it.

Strategy 4 — Late chunking

Normal flow: *chunk → embed each chunk alone*. A chunk saying "*It* was founded in 1998" has no idea what "it" is. Late chunking reverses the order: run the **whole document** through a long-context embedding model to get one contextualized vector *per token*, then for each chunk span **average (mean-pool) the token vectors** inside that span. Every token vector was computed while "seeing" the full document, so the chunk vector carries global context.

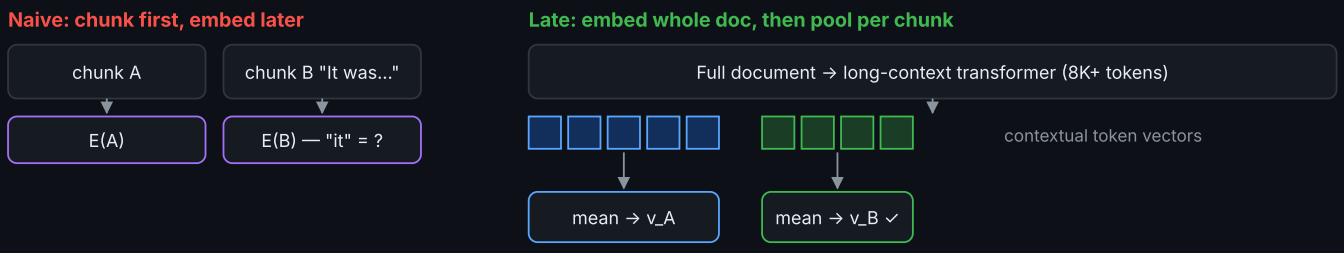


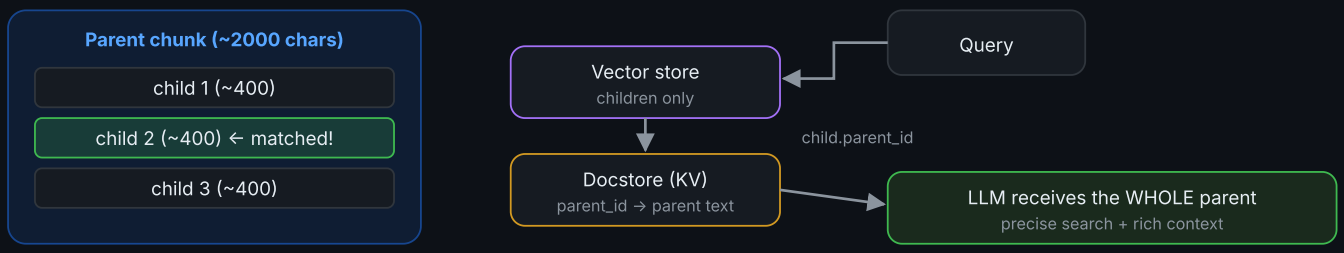
Fig 3.4 — Late chunking needs an embedding model that exposes token-level outputs and a long context window (e.g. Jina embeddings); it's not a drop-in option for API-only models.

3.4 Advanced Retrieval-Time Patterns

Parent–Child (Small-to-Big) Retriever

Conflict: **small chunks search better** (sharp vectors) but **big chunks answer better** (full context). Solution: index small child chunks for search, but when a child matches, return its larger **parent** to the

LLM.




```python
# LangChain >= 1.0: legacy retrievers moved to the langchain-classic package
from langchain_classic.retrievers import ParentDocumentRetriever
from langchain_core.stores import InMemoryStore      # production: Redis / Postgres-backed store

retriever = ParentDocumentRetriever(
    vectorstore=vector_store,                    # holds child vectors
    docstore=InMemoryStore(),                    # holds parent text
    child_splitter=RecursiveCharacterTextSplitter(chunk_size=400),
    parent_splitter=RecursiveCharacterTextSplitter(chunk_size=2000),
)
retriever.add_documents(docs)
```

VERSION NOTE

LangChain 1.0 slimmed the `langchain` package down to agents; older helpers like `ParentDocumentRetriever`, `EnsembleRetriever` and compression retrievers now live in `langchain-classic` (`uv add langchain-classic`). On an older 0.3.x install the same classes import from `langchain.retrievers`. If an import fails, check which major version you have.

Contextual Compression

After retrieval, pass each chunk through a cheap LLM (or an embeddings filter) that **keeps only sentences relevant to the query**. Fewer tokens, less noise. Cost: one extra LLM call per chunk → added latency. Use when chunks are long and noisy; skip when latency matters most.

```python
from langchain_classic.retrievers import ContextualCompressionRetriever
from langchain_classic.retrievers.document_compressors import LLMChainExtractor

compressor = LLMChainExtractor.from_llm(cheap_llm)
compressed_retriever = ContextualCompressionRetriever(
    base_compressor=compressor, base_retriever=vector_store.as_retriever(search_kwargs={"k": 8}),
)
```

3.5 How to actually choose chunk settings

1. Build a small eval set: 30–50 real questions + which document/section answers each.
2. Try 3–4 configs (e.g. 256 / 512 / 1024 tokens, 10–15% overlap).
3. Measure **Hit@k** (is the correct chunk in the top-k?) and **MRR** (how high is it ranked?).
4. Pick the winner; only then try semantic / parent-child / contextual retrieval and keep them only if numbers improve.

SEEDHI BAAT

Chunk size ka koi "best number" nahi hota. Short FAQ-type answers → chhote chunks. Long legal / technical reasoning → bade chunks ya parent-child. Andaza mat lagao, 40 sawaalon pe test karo.

04 Embeddings & Vector Stores

Text → numbers, jahan "matlab mein paas" = "space mein paas".

4.1 Embedding model vs Chat model

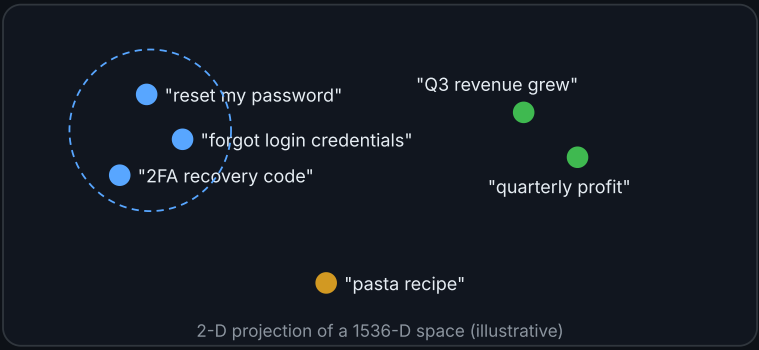
CHAT MODEL

Text → Text. Generates tokens one at a time. Used for reasoning and writing the answer. Expensive per call. Examples: Claude, GPT.

EMBEDDING MODEL

Text → fixed-length vector $\mathbb{R}^d$. One forward pass, no generation. Cheap and fast. Used only for similarity search. Examples: text-embedding-3-small, BGE.

Embedding models are trained (contrastively) so that a question and the passage that answers it end up close together, and unrelated texts end up far apart.



- Meaning clusters, not spelling: "forgot credentials" ≈ "reset password" even with zero shared words.
- Weakness: exact strings "ERR-503" vs "ERR-505" look nearly identical to an embedding model → solved by BM25 hybrid search (Ch 5).

4.2 Dimensions & Model Choice

Model	Dims (d)	Speed	Notes
OpenAI text-embedding-3-small	1,536 (shrinkable)	High	Best default price/quality. `dimensions=` param lets you request 512 or 256.
OpenAI text-embedding-3-large	3,072 (shrinkable)	Medium	Highest quality; 2× storage of small at full size.
Google Gemini embeddings	768 (text-embedding-004) / up to 3,072 (gemini-embedding-001)	High	Free tier available; newer model supports choosing 768/1536/3072.
BGE-small-en-v1.5 (open source)	384	Very high (local)	Runs on CPU, no API cost; weaker on hard domains.

STORAGE MATH

1 vector = d × 4 bytes (float32). 1M chunks × 1536 dims × 4 B = **6.1 GB** of raw vectors, plus HNSW graph overhead. At 512 dims: 2.0 GB. This is why dimension and quantization choices (Ch 8) matter at scale.

CORRECTION

The notes list "Gemini Embeddings — 768". That's true for the older `text-embedding-004`; the current `gemini-embedding-001` defaults to 3,072 and can be truncated (Matryoshka-style), like OpenAI's v3 models. Always check the model card for current dims and pricing.

4.3 Distance Metrics

Cosine similarity $\cos(\mathbf{u},\mathbf{v}) = (\mathbf{u}\cdot\mathbf{v}) / (\|\mathbf{u}\|\,\|\mathbf{v}\|) = \Sigma\, u_i v_i / (\sqrt{\Sigma u_i^2} \cdot \sqrt{\Sigma v_i^2})$ range $[-1, 1]$, higher = more similar

Euclidean (L2) $d(\mathbf{u},\mathbf{v}) = \sqrt{\Sigma(u_i - v_i)^2}$ range $[0, \infty)$, lower = more similar

Dot product $\mathbf{u}\cdot\mathbf{v} = \Sigma\, u_i v_i$ higher = more similar; rewards long vectors

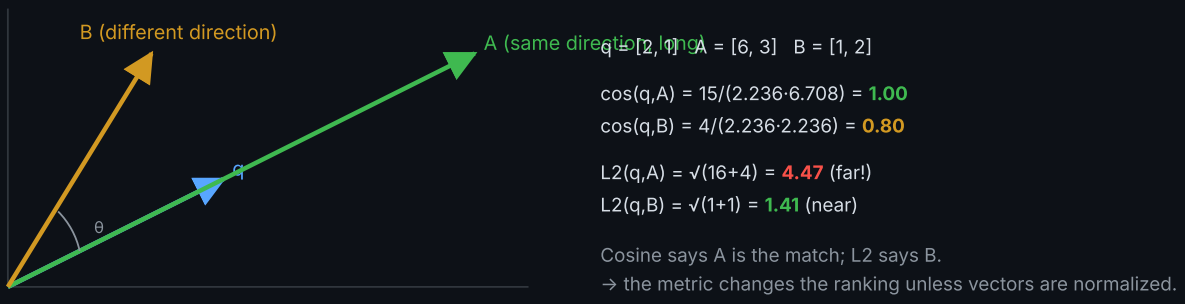


Fig 4.1 — Cosine cares only about direction (topic); L2 also cares about length.

SEEDHI BAAT — KAUNSA USE KAREIN?

OpenAI, Gemini, BGE (normalize ke saath) sab **unit-length** vectors dete hain ($\|v\| = 1$). Tab teeno metrics same ranking dete hain, kyunki $\|u-v\|^2 = 2 - 2\cdot\cos(u,v)$. Isliye: **cosine** use karo (safe default), ya normalized vectors pe **dot product** (sabse fast, ek division kam).

4.4 Chroma: Persistence, Distance & Metadata Filters

PYTHON

```python
from langchain_chroma import Chroma                # maintained package (not langchain_community)
from langchain_openai import OpenAIEmbeddings

embeddings = OpenAIEmbeddings(model="text-embedding-3-small")

vector_store = Chroma(
    collection_name="tech_documentation",
    embedding_function=embeddings,
    persist_directory="./chroma_db",
    collection_metadata={"hnsw:space": "cosine"},  # Chroma's default is L2!
)
vector_store.add_documents(chunks, ids=[c.metadata["chunk_id"] for c in chunks])

results = vector_store.similarity_search_with_score(
    "How do I configure database connection pooling?",
    k=3,
    filter={"$and": [{"topic": "database"}, {"version": {"$gte": 2}}]},
)
for doc, distance in results:          # NOTE: a DISTANCE — lower is better
    print(f"[dist={distance:.4f}] {doc.page_content[:80]}… | {doc.metadata}")
```

CORRECTION

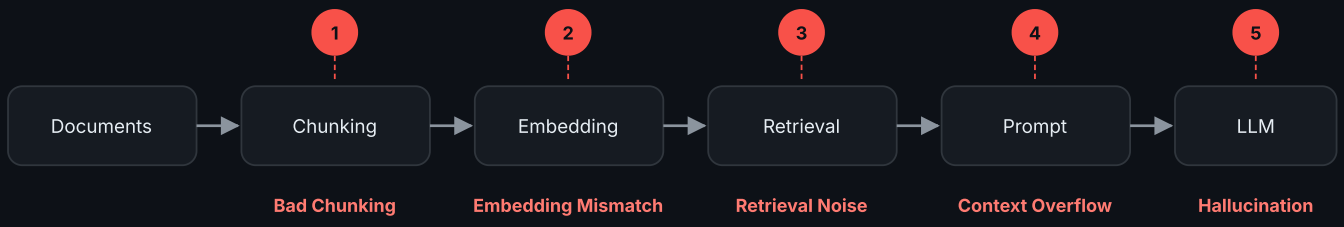
The notes print the value as `Score`, implying higher = better. In Chroma, `similarity_search_with_score` returns a **distance** (cosine distance = 1 − cos), so **lower is better**. If you want a 0–1 "higher is better" number use `similarity_search_with_relevance_scores`. Also, the collection uses L2 unless you set `hnsw:space` at creation; it can't be changed later without rebuilding.

PRE-FILTER VS POST-FILTER

Metadata filters are what make multi-tenant RAG possible: `filter={"tenant_id": user.org}` ensures a user can **never** retrieve another company's chunks. Enforce this filter server-side from the auth token — never from user input.

05 The Five Production Failure Modes

Demo pe sab chalta hai; production mein inhi 5 jagah se RAG tootta hai. Interview mein sabse zyada yahi poocha jaata hai.



Each failure upstream compounds downstream: a bad chunk can never be fixed by a better LLM.

Fig 5.1 — Where each failure mode lives in the pipeline.

#	Failure	Symptom you'll see	Root cause	Fixes
1	Bad chunking	Right doc retrieved but answer incomplete / wrong entity	Boundaries cut through meaning	Recursive / structure-aware split, overlap, parent-child, contextual retrieval
2	Embedding mismatch	Relevant doc exists but never appears in top-k	User vocabulary ≠ document vocabulary; exact codes	Query rewriting, multi-query, HyDE, hybrid BM25, better/domain embeddings
3	Retrieval noise	Top-k full of "kind of related" chunks	Pure vector similarity is fuzzy	Hybrid search + RRF, reranker, metadata filters, similarity threshold
4	Context overflow	Token errors, high bills, slow responses, ignored middle chunks	Unbounded k / chunk size / chat history	Token budgeting, rerank-then-truncate, history summarization, model routing
5	Hallucination	Fluent answer not supported by sources	Model fills gaps from its own knowledge	Strict grounding prompt, citations, refusal string, faithfulness check / self-correction

ABOUT "90% OF NAIVE RAG FAILS"

That figure in the notes is a common blog-post claim, not a measured statistic — don't quote it as fact in an interview. Say instead: "most failures I've seen trace back to retrieval, not generation."

5.1 Failure Mode 2 — Embedding Mismatch

User asks "*my laptop won't turn on*"; the manual says "*device fails to power up after firmware update*". Semantically similar, but short queries with different vocabulary often embed far from the answer. Fixes that work at query time:

- **Query rewriting:** an LLM rewrites the user's casual query into documentation language.
- **Multi-query:** generate 3–5 paraphrases, retrieve for each, merge with RRF. Raises recall.
- **HyDE (Hypothetical Document Embeddings):** ask the LLM to write a *fake answer*, embed *that*, and search with it. Answers look like documents, so they land near real documents.
- **Hybrid BM25** for exact identifiers (next section).

CORRECTION

The notes list "high-dimension embeddings" as the fix. More dimensions add nuance but don't fix vocabulary mismatch or exact-code lookup; query transformation and hybrid search are the real fixes.

5.2 Failure Mode 3 — Hybrid Search (BM25 + Vector) with RRF

Vector search understands *meaning* but is weak at *exact tokens*: error codes (`ERR-503`), SKUs (`SQ7742X`), function names, acronyms, people's names. BM25 is the opposite. Run both and fuse.



Fig 5.2 — Hybrid retrieval. D3 (the doc that literally mentions ERR-503) was only #3 for vectors; fusion lifts it to #2.

BM25 — reading the formula term by term

$$\mathrm{BM25}(q, d) = \Sigma_{\text{terms } q_i \text{ in } q} \; \mathrm{IDF}(q_i) \cdot [\, f(q_i,d)\cdot(k_1+1) \,] \,/\, [\, f(q_i,d) + k_1\cdot(1 - b + b\cdot|d|/\mathrm{avgdl}) \,]$$

Piece	Meaning	Effect
$\mathrm{IDF}(q_i)$	Inverse document frequency — how rare the term is across the corpus	"ERR-503" (rare) scores high; "the", "error" (common) score near zero
$f(q_i, d)$	How many times the term occurs in document d	More mentions → higher score…
k_1 (≈1.2–2.0)	Term-frequency saturation	…but with diminishing returns: 10 mentions isn't 10× better than 1
b (≈0.75), $\|d\|/\mathrm{avgdl}$	Length normalization	Long docs naturally contain more words, so they're penalized a bit

RRF — worked example

$$\mathrm{RRF}(d) = \Sigma_{\text{retrievers } m} \; 1 \,/\, (k + \mathrm{rank}_m(d)), \quad k = 60$$

Doc	Vector rank	BM25 rank	Calculation	RRF score	Final
D1	1	2	1/61 + 1/62	0.03252	**1**
D3	3	1	1/63 + 1/61	0.03227	**2**
D2	2	—	1/62	0.01613	3
D4	—	3	1/63	0.01587	4

SEEDHI BAAT — RRF KYUN?

BM25 score 0–25 jaisa kuch bhi ho sakta hai, cosine 0–1. In dono ko seedha add karna = seb + santre. RRF score ko ignore karke sirf **rank** dekhta hai, toh normalization ki zarurat hi nahi. Jo doc **dono** lists mein upar hai woh jeetta hai. k = 60 isliye ki rank 1 aur rank 5 ka fark zyada bada na ho — ek retriever akela dominate na kare.

PYTHON

```python
from langchain_community.retrievers import BM25Retriever
from langchain_core.documents import Document

def reciprocal_rank_fusion(results_list: list[list[Document]], k: int = 60) -> list[Document]:
    """Fuse several ranked lists. Key by a stable chunk id (fallback: content)."""
    scores: dict[str, dict] = {}
    for docs in results_list:
        for rank, doc in enumerate(docs, start=1):
            key = doc.metadata.get("chunk_id") or doc.page_content
            entry = scores.setdefault(key, {"doc": doc, "score": 0.0})
            entry["score"] += 1.0 / (k + rank)
    ranked = sorted(scores.values(), key=lambda x: x["score"], reverse=True)
    return [e["doc"] for e in ranked]

bm25 = BM25Retriever.from_documents(chunks, k=10)          # in-memory; rebuild when corpus changes
vector = vector_store.as_retriever(search_kwargs={"k": 10})

def hybrid_search(query: str, top_n: int = 5) -> list[Document]:
    return reciprocal_rank_fusion([vector.invoke(query), bm25.invoke(query)])[:top_n]
```

PRODUCTION NOTE

`BM25Retriever` lives in RAM and must be rebuilt on every corpus change — fine for thousands of chunks, not for millions. At scale use a store with native hybrid search: Postgres (pgvector + full-text `tsvector`), Elasticsearch/OpenSearch, Weaviate, Qdrant or Pinecone sparse-dense. LangChain's `EnsembleRetriever` (in `langchain_classic`) implements weighted RRF for you.

Add a Reranker (the biggest cheap win)

Embedding search is a **bi-encoder**: query and document are encoded separately, so the model never sees them together. A **cross-encoder reranker** reads *(query, chunk)* as one input and outputs a relevance score — far more accurate, but too slow to run over the whole corpus. So: retrieve wide (top 20–50 via hybrid), rerank, keep the top 3–5.



Fig 5.3 — Two-stage retrieval: recall first, precision second. Options: Cohere Rerank, Voyage rerank, BGE-reranker / ms-marco cross-encoders (self-hosted).

PYTHON

```python
from sentence_transformers import CrossEncoder
reranker = CrossEncoder("BAAI/bge-reranker-base")

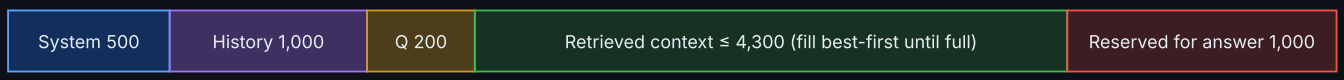
def rerank(query: str, docs: list[Document], top_n: int = 4) -> list[Document]:
    scores = reranker.predict([(query, d.page_content) for d in docs])
    ranked = sorted(zip(docs, scores), key=lambda x: x[1], reverse=True)
```

```
    return [d for d, _ in ranked[:top_n]]

final_docs = rerank(query, hybrid_search(query, top_n=30))
```

5.3 Failure Mode 4 — Context Overflow & Token Budgets

Every token costs money and latency, and context windows are finite. Worse, models attend best to the start and end of long prompts ("**lost in the middle**"), so stuffing 30 chunks can make answers *worse*. Budget the prompt explicitly:



Example budget: 7,000 tokens total per request

PYTHON

```python
import tiktoken
ENC = tiktoken.get_encoding("cl100k_base")       # use your model's tokenizer when available

def count_tokens(text: str) -> int:
    return len(ENC.encode(text))

def fit_context(docs: list[Document], budget: int) -> list[Document]:
    """Docs arrive best-first (after rerank). Keep adding until the budget is spent."""
    kept, used = [], 0
    for d in docs:
        t = count_tokens(d.page_content)
        if used + t > budget:
            break
        kept.append(d); used += t
    return kept

class BudgetedLLM:
    def __init__(self, llm, max_input_tokens: int = 6000):
        self.llm, self.max_input = llm, max_input_tokens
    def invoke(self, prompt_text: str):
        n = count_tokens(prompt_text)
        if n > self.max_input:
            raise ValueError(f"Prompt is {n} tokens > budget {self.max_input}")
        return self.llm.invoke(prompt_text)
```

CORRECTION

The notes estimate tokens as `words × 1.3`. It's okay for English prose but badly wrong for code, URLs, JSON and Hindi/Devanagari text (which can use several tokens per word). Use a real tokenizer (`tiktoken`, or the provider's token-counting endpoint) for any budget you enforce. Also: rejecting the whole request is harsh — prefer **truncating context** (as `fit_context` does) and summarizing old chat history.

Dynamic model routing: send short, simple questions to a small cheap model and long or complex ones (many chunks, multi-step reasoning) to a bigger model. A simple router can use prompt length, a keyword classifier, or a tiny LLM call.

5.4 Failure Mode 5 — Hallucination

1. **Grounded prompt** with a fixed refusal sentence (Ch 1.4).

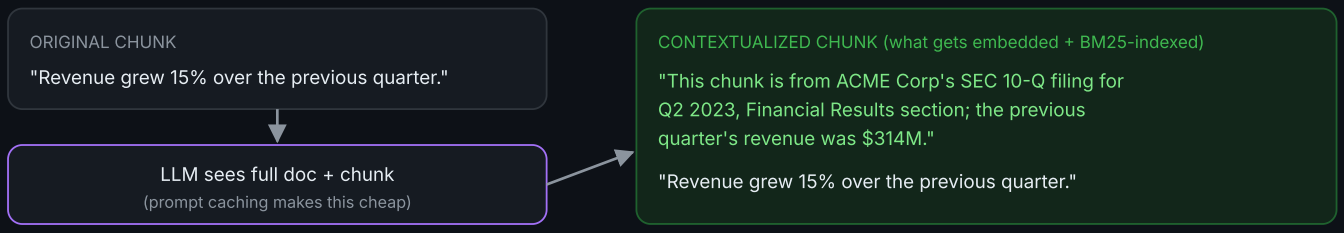
2. **Citations required** per claim — uncited sentences are suspect and can be stripped or flagged.
3. **Retrieval threshold:** if the best chunk is below a relevance cutoff, don't call the LLM at all; return the refusal. (No evidence → no answer.)
4. **Faithfulness check:** a second, cheap LLM call asks "is every claim in ANSWER supported by CONTEXT? yes/no + unsupported claims". On "no", regenerate or refuse. This is the core of self-correcting RAG (Ch 6.2).
5. **Lower temperature** (0–0.2) for factual Q&A.

06 Advanced RAG Architectures

Basic pipeline ke upar ke patterns — kab chahiye aur kab overkill hai.

6.1 Contextual Retrieval (Anthropic, 2024)

Problem: a chunk like "*Revenue grew 15% over the previous quarter*" doesn't say *which company* or *which quarter*. Its embedding and its BM25 terms can't match a query like "ACME Q2 2023 revenue growth". Fix: before indexing, ask an LLM to write a 1–2 sentence **context prefix** for each chunk using the full document, and prepend it. Index the prefixed text in **both** the vector store and BM25.



Configuration (Anthropic's benchmark, top-20 retrieval)	Failure rate reduction
Contextual Embeddings only	35%
Contextual Embeddings + Contextual BM25	49%
Contextual Embeddings + Contextual BM25 + Reranking	67%

CORRECTION

The notes say Contextual Retrieval "reduces retrieval failures by 49% alone". In Anthropic's published results, 49% already includes contextual BM25 (hybrid); contextual embeddings alone gave ~35%; adding reranking gave 67%.

PYTHON

```python
CONTEXT_PROMPT = """<document>
{doc}
</document>
Here is the chunk we want to situate within the whole document:
<chunk>
{chunk}
</chunk>
Give a short succinct context (1-2 sentences) to situate this chunk within the overall
document for the purposes of improving search retrieval. Answer only with the context."""

def contextualize(full_doc: str, chunk_text: str, llm) -> str:
    prefix = llm.invoke(CONTEXT_PROMPT.format(doc=full_doc, chunk=chunk_text)).content.strip()
    return f"{prefix}\n\n{chunk_text}"
```

COST TRICK

The full document is identical for every chunk of that document — put it first in the prompt and use **prompt caching** so you pay full price for it once, then a fraction for each subsequent chunk. Use a small, cheap model for this step; it's a summarization task.

6.2 Agentic / Self-Correcting RAG with LangGraph

Linear RAG retrieves once and hopes. Agentic RAG turns the pipeline into a **state machine** that can judge its own retrieval, rewrite the query and retry, and fall back safely — with a hard cap on retries so it can't loop forever.

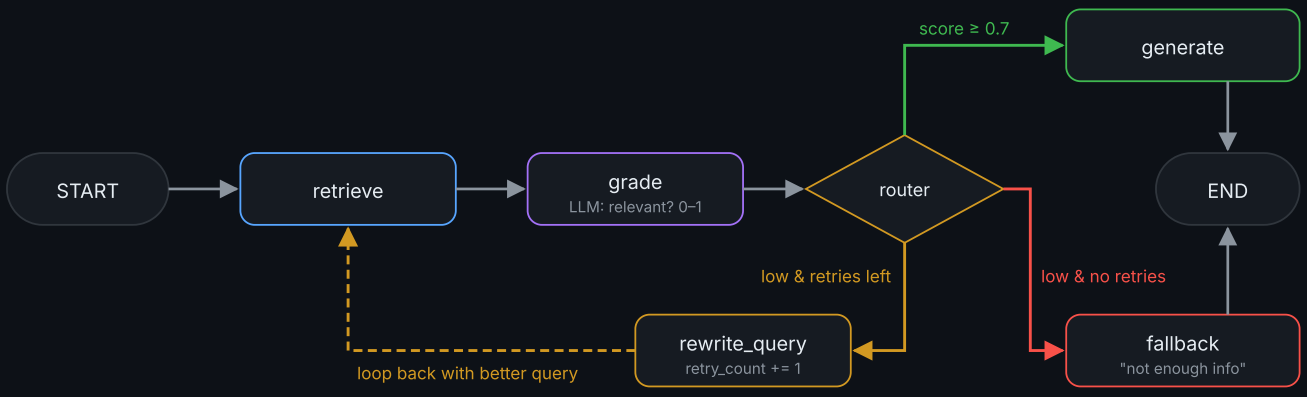


Fig 6.1 — Self-correcting retrieval loop. The retry cap (max_retries) guarantees termination.

The notes' code referenced `generate_response` and `fallback_response` nodes that were never added, and never compiled the graph. Complete, runnable version:

PYTHON

```python
from typing import TypedDict
from pydantic import BaseModel, Field
from langchain_core.documents import Document
from langgraph.graph import StateGraph, START, END

class RAGState(TypedDict):
    question: str          # original user question (never overwritten)
    query: str             # current search query (may be rewritten)
    documents: list[Document]
    relevance: float
    retry_count: int
    max_retries: int
    generation: str

class Grade(BaseModel):
    score: float = Field(description="0-1: how well the documents can answer the question")

grader = llm.with_structured_output(Grade)        # forces JSON {score: float}

def retrieve(state: RAGState):
    return {"documents": rerank(state["query"], hybrid_search(state["query"], 30))}

def grade(state: RAGState):
    ctx = format_docs_with_sources(state["documents"])
    g = grader.invoke(f"Question: {state['question']}\n\nDocuments:\n{ctx}\n\nScore relevance 0-1.")
    return {"relevance": g.score}

def route(state: RAGState) -> str:
    if state["relevance"] >= 0.7:
        return "generate"
    return "rewrite" if state["retry_count"] < state["max_retries"] else "fallback"

def rewrite(state: RAGState):
    new_q = llm.invoke(
        f"Rewrite this search query to better match technical documentation. "
        f"Return only the query.\nOriginal question: {state['question']}\nLast query:
```

```
{state['query']}"
    ).content.strip()
    return {"query": new_q, "retry_count": state["retry_count"] + 1}

def generate(state: RAGState):
    msgs = rag_prompt.invoke({"context": format_docs_with_sources(state["documents"]),
                              "question": state["question"]})
    return {"generation": llm.invoke(msgs).content}

def fallback(state: RAGState):
    return {"generation": "I don't have enough information to answer this question."}

g = StateGraph(RAGState)
for name, fn in [("retrieve", retrieve), ("grade", grade), ("rewrite", rewrite),
                 ("generate", generate), ("fallback", fallback)]:
    g.add_node(name, fn)
g.add_edge(START, "retrieve")
g.add_edge("retrieve", "grade")
g.add_conditional_edges("grade", route, {"generate": "generate", "rewrite": "rewrite", "fallback":
"fallback"})
g.add_edge("rewrite", "retrieve")
g.add_edge("generate", END)
g.add_edge("fallback", END)
app_graph = g.compile()

result = app_graph.invoke({"question": q, "query": q, "documents": [], "relevance": 0.0,
                           "retry_count": 0, "max_retries": 2, "generation": ""})
print(result["generation"])
```

SEEDHI BAAT

Node = ek function jo state leta hai aur state ka *partial update* (dict) return karta hai. Edge = "iske baad kaun". Conditional edge = router function ka return value decide karta hai agla node. `question` aur `query` alag rakho — rewrite sirf search query badalta hai, final answer hamesha user ke original sawaal ka hona chahiye.

COST OF AGENTIC RAG

Each loop adds 2–3 LLM calls (grade + rewrite + re-retrieve). Worst case with max_retries=2: ~7 LLM calls per question. Log the retry rate — if most queries loop, fix retrieval itself instead of relying on the loop.

6.3 Graph RAG (Microsoft)

Vector RAG retrieves passages that *look like* the question. It fails on **multi-hop** questions where the answer needs facts from several documents chained together: "Which products are affected by suppliers in regions managed by Executive X?" No single chunk contains that chain.

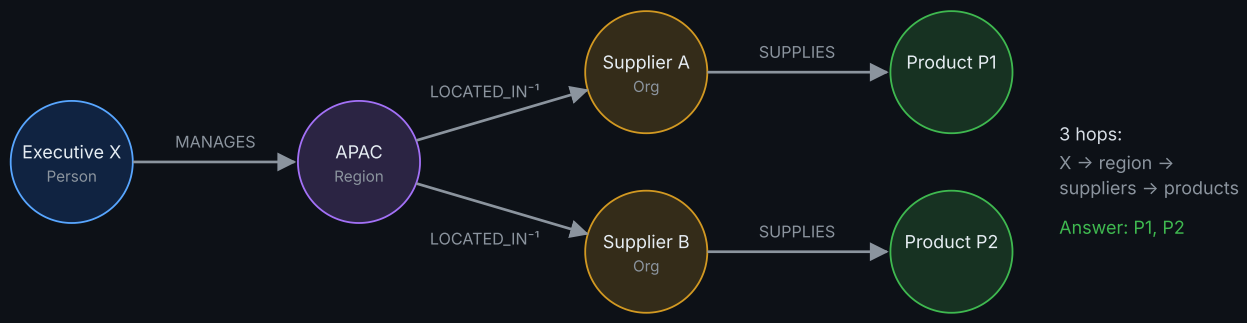


Fig 6.2 — An LLM extracts entities (nodes) and relations (edges) from every chunk at index time; queries then traverse the graph.

LOCAL SEARCH

Find entities named in the question, walk their neighbourhood (1–3 hops), collect related entities, relations and source chunks → LLM answers. For specific, relational questions.

GLOBAL SEARCH

The graph is clustered into **communities** (Leiden algorithm) and each gets an LLM-written summary at index time. Broad questions ("main risks across all facilities?") are answered map-reduce style over these summaries.

WHEN NOT TO USE IT

Indexing is expensive (LLM extraction over every chunk + community summaries) and graphs go stale as documents change. Use Graph RAG only when your questions are genuinely relational or corpus-wide summaries; for normal fact lookup, hybrid + rerank wins on cost.

6.4 Multimodal RAG (ColPali)

Text extraction destroys tables (columns misalign), charts (become nothing) and layout cues. ColPali skips OCR: it treats each **page as an image** and embeds it with a vision-language model (PaliGemma backbone).

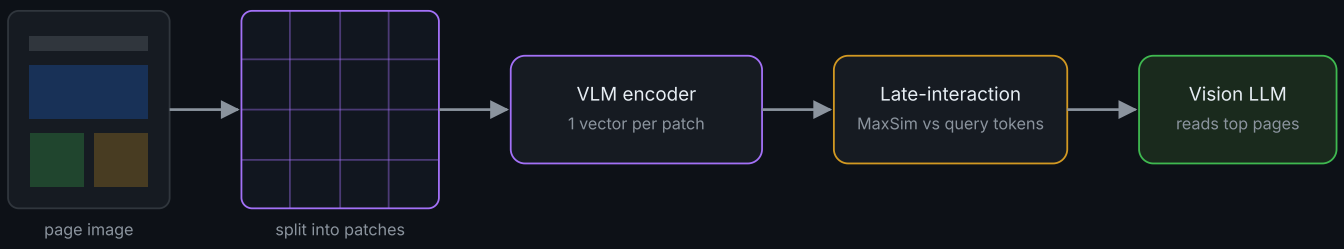


Fig 6.3 — ColPali pipeline. Score = Σ over query tokens of max similarity with any page patch ("MaxSim", from ColBERT).

Trade-off: ColPali stores roughly a thousand vectors *per page* instead of one per chunk, so storage and search need a multi-vector-capable store (Vespa, Qdrant multivector, etc.), and answering requires a vision-capable LLM. Use it for financial statements, scanned forms, slide decks, engineering drawings — anywhere layout carries meaning.

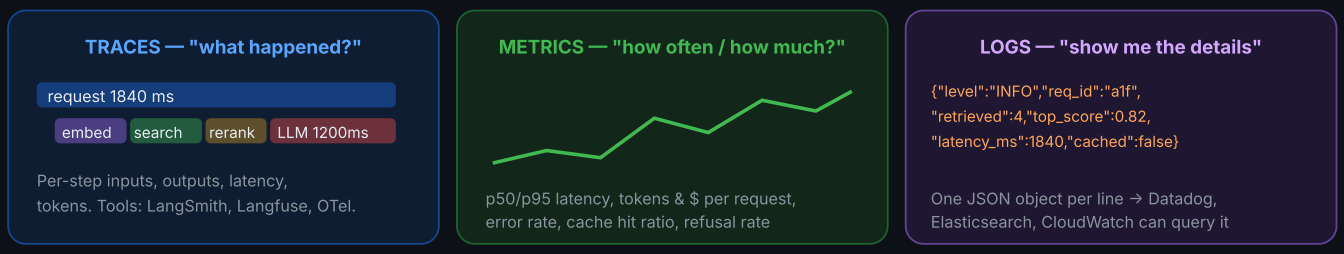
6.5 Which pattern when?

Situation	Start with	Add if needed
Docs, FAQs, knowledge base	Recursive chunks + hybrid + rerank	Contextual retrieval
Lots of IDs, codes, names	Hybrid (BM25 is essential)	Metadata filters
Vague / conversational users	Query rewriting, multi-query	Agentic loop
Relationship / multi-hop questions	Graph RAG (local)	Hybrid graph + vector
"Summarize themes across everything"	Graph RAG (global) or map-reduce	—
Tables, charts, scans	Unstructured loader with table extraction	ColPali

07 Observability, Tracing & Evaluation

LLM systems stochastic hote hain — bina tracing ke debugging = andhere mein teer.

7.1 The Three Pillars



7.2 What to record for every RAG request

Field	Why
request_id / thread_id / user or tenant id	Join traces, logs and user complaints
original query, rewritten query	See what was actually searched
retrieved chunk ids + scores (pre- and post-rerank)	Most bad answers are bad retrieval — this proves it
prompt tokens, completion tokens, model name, cost	Cost dashboards; catch runaway prompts
latency per stage	Find the slow step (usually the LLM, sometimes rerank)
refusal flag, retry count, cache hit, fallback used	Health signals for retrieval coverage
user feedback (👍/👎)	Cheapest source of labelled eval data

PRIVACY

Logs with raw queries and chunks may contain PII. Mask before logging, set retention limits, and restrict access — same rules as your database.

7.3 LangSmith + Structured JSON Logging

PYTHON

```python
# .env  (never hard-code keys in source)
# LANGSMITH_TRACING=true
# LANGSMITH_API_KEY=lsv2_...
# LANGSMITH_PROJECT=production-rag-api
# (older LANGCHAIN_TRACING_V2 / LANGCHAIN_API_KEY names still work)

import json, logging
from datetime import datetime, timezone

class JSONLogFormatter(logging.Formatter):
    def format(self, record: logging.LogRecord) -> str:
        obj = {
            "ts": datetime.now(timezone.utc).isoformat(),
            "level": record.levelname,
            "msg": record.getMessage(),
            "module": record.module,
```

```
            "func": record.funcName,
        }
        obj.update(getattr(record, "extra_data", {}))
        if record.exc_info:
            obj["exc"] = self.formatException(record.exc_info)
        return json.dumps(obj, ensure_ascii=False)

logger = logging.getLogger("rag")
_h = logging.StreamHandler(); _h.setFormatter(JSONLogFormatter())
logger.addHandler(_h); logger.setLevel(logging.INFO)

logger.info("rag_request", extra={"extra_data": {
    "req_id": "a1f", "top_score": 0.82, "n_docs": 4, "latency_ms": 1840}})
```

CORRECTION

The notes hard-code `LANGCHAIN_API_KEY = "ls__your_api_key"` inside Python — keys belong in environment variables / a secrets manager. `datetime.utcnow()` is deprecated since Python 3.12; use `datetime.now(timezone.utc)`. Added exception capture to the formatter.

7.4 Evaluation — how do you know it's good?

Observability tells you what happened; **evaluation** tells you if it was right. Evaluate retrieval and generation **separately**, because the fix differs.

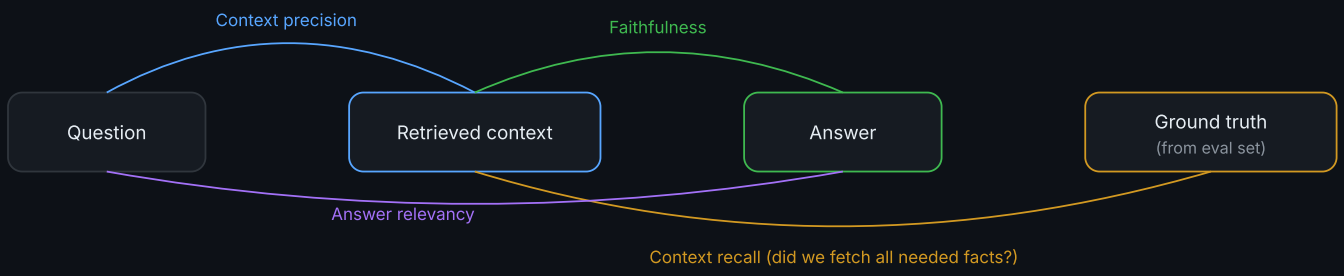


Fig 7.1 — The four standard RAG metrics (as popularised by RAGAS) and which pair each compares.

Metric	Question it answers	If low, fix…
Context precision	Are the retrieved chunks relevant, and ranked high?	Reranker, filters, hybrid
Context recall	Did retrieval find everything needed?	Chunking, k, query rewriting, embeddings
Faithfulness	Is every claim in the answer supported by context?	Prompt grounding, temperature, model
Answer relevancy	Does the answer address the question?	Prompt, model

Workflow: keep a golden set of 50–200 Q&A pairs; run it in CI on every change to chunking, prompt, model or index settings; block deploys that regress. Metrics are usually computed by an "LLM-as-judge" — spot-check the judge against human labels before trusting it.

08 Vector Index Tuning & Cost

1M vectors pe brute-force search = har query pe 1M dot products. HNSW isko milliseconds mein karta hai — thodi accuracy ke badle.

8.1 How HNSW works

Hierarchical Navigable Small World is a multi-layer graph. The top layer has few nodes with long-range links (like highways); each lower layer has more nodes and shorter links; layer 0 contains every vector. Search starts at the top, greedily moves to the neighbour closest to the query, drops a layer, repeats — like zooming in on a map: country → city → street.

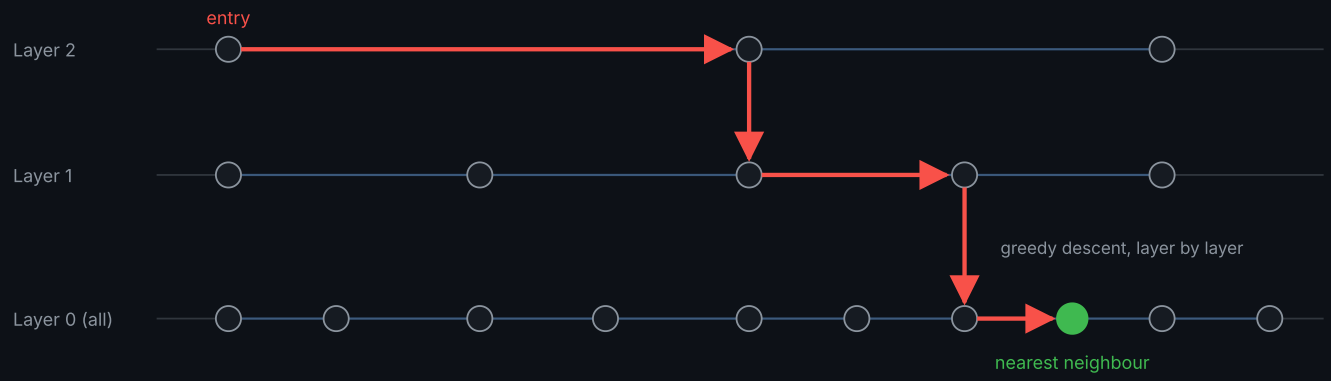


Fig 8.1 — Search path (red). Only ~log(N) nodes are visited instead of all N.

8.2 The three knobs

Parameter	When it applies	Meaning	Higher value →	pgvector default
M	Build time	Max links per node per layer	Better recall, more RAM, slower build	16
ef_construction	Build time	Candidate list size while inserting	Better graph quality, slower build (no query cost)	64
ef_search	Query time	Candidate list size while searching (must be ≥ k)	Better recall, slower queries	40

SEEDHI BAAT

M = har ghar se kitni sadkein (zyada sadkein = rasta galat hone ka chance kam, par map bada). **ef_search** = dhoondhte waqt kitne options saath mein yaad rakhte ho. Tuning ka tareeka: M=16 rakho, phir ef_search 40 → 100 → 200 badhao jab tak recall@10 (exact search ke against) ~95–99% na ho jaye, aur p95 latency dekhte raho.

SQL

```sql
CREATE EXTENSION IF NOT EXISTS vector;

CREATE TABLE document_embeddings (
  id         bigserial PRIMARY KEY,
  tenant_id  text NOT NULL,
  content    text NOT NULL,
```

```
  metadata   jsonb,
  embedding  vector(1536)
);

-- Build-time params
CREATE INDEX ON document_embeddings
  USING hnsw (embedding vector_cosine_ops) WITH (m = 16, ef_construction = 64);

-- Query-time effort (per session / transaction)
SET hnsw.ef_search = 100;

-- <=> is cosine DISTANCE in pgvector (smaller = closer)
SELECT id, content, 1 - (embedding <=> $1) AS cosine_sim
FROM document_embeddings
WHERE tenant_id = $2
ORDER BY embedding <=> $1
LIMIT 5;
```

FILTER + ANN GOTCHA

With a selective `WHERE` filter, HNSW first finds ef_search candidates and *then* filters — you may get fewer than k rows. Fixes: raise ef_search, enable iterative scans (`SET hnsw.iterative_scan = relaxed_order;` in pgvector ≥ 0.8), or partition by tenant.

8.3 Cost by scale

Corpus	Pinecone serverless	Managed PGVector	Self-hosted PGVector	Sensible choice
~500K vectors	~$20–30 / mo	~$32–35 / mo	~$15–20 / mo	Whatever is zero-ops (Pinecone / Supabase); or pgvector in the Postgres you already run
~5M vectors	~$400 / mo	~$100 / mo	~$75 / mo	Managed Postgres + pgvector
~50M vectors	$1,500+ / mo	~$400 / mo	~$300 / mo	Self-hosted / dedicated instances, or a purpose-built engine

TREAT THESE AS ROUGH, DATED ESTIMATES

These figures come from your notes; real costs depend heavily on query volume, dimensions, replicas and region, and vendor pricing changes often. In an interview say "managed serverless is cheapest at small scale and zero-ops; past a few million vectors with steady traffic, Postgres/self-hosting usually wins on cost" — and check current pricing pages before a real decision. Also count engineering time: self-hosting isn't free.

8.4 Cost optimization levers — with the right math

Lever	How	Memory saving	Quality impact
Dimension reduction	OpenAI v3: `OpenAIEmbeddings(dimensions=512)`	1536 → 512 = **66.7%**	Small drop; v3 models are trained for truncation (Matryoshka)
Scalar quantization	float32 → int8	**75%** (4 B → 1 B)	Usually ~1% recall loss
Half precision	float32 → float16 (pgvector `halfvec`)	**50%**	Negligible
Binary quantization	1 bit per dim + rerank with full vectors	**~97%** (32×)	Needs rescoring step
Exact cache	hash(normalized query) → answer	Skips the whole pipeline for repeated questions	
Semantic cache	embed query; if cos ≥ 0.95 to a cached query → reuse	Higher hit rate but risk of wrong answers ("cancel order" vs "cancel subscription") — keep threshold high, scope per tenant	

CORRECTION

Notes said 1536→512 saves "30–60%" — it's 66.7% (512/1536 = ⅓ kept). Float32→Int8 saves exactly 75%, not "50–75%" (50% is float16). Exact caching should use a proper hash (SHA-256) of a normalized query; MD5 works for cache keys but don't use it for anything security-related.

09 FastAPI, Security & Deployment

Model ko API ke peeche daalna — rate limits, security, caching, fallbacks ke saath.

9.1 Request lifecycle

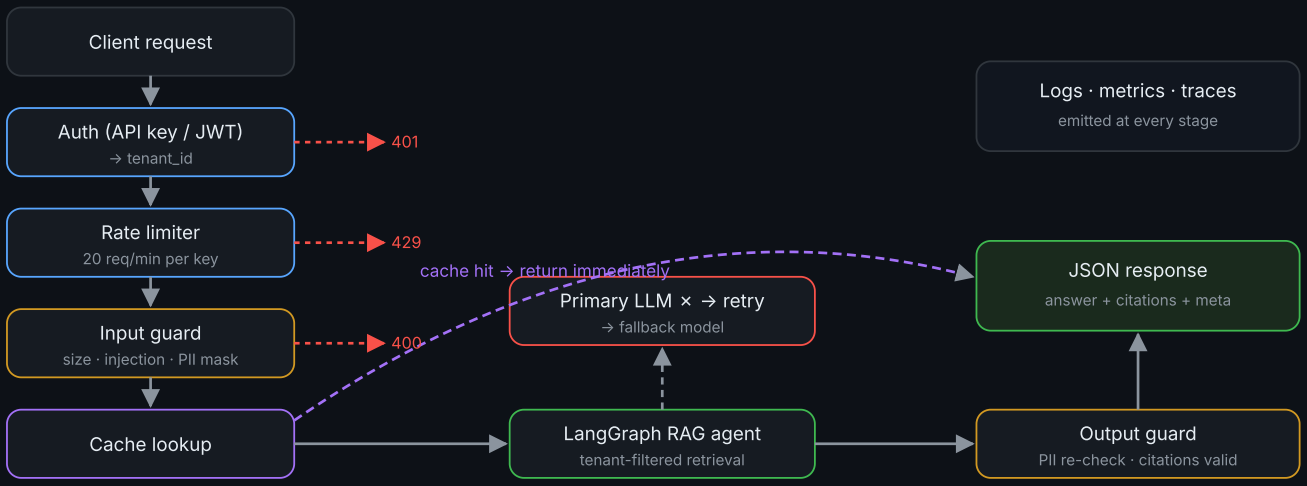


Fig 9.1 — Cheap checks first (auth, rate limit, validation), expensive work (LLM) last.

9.2 Security — what's real and what's theatre

Threat	Naive defence (notes)	Production defence
Direct prompt injection ("ignore previous instructions")	2 regex patterns	Regex = cheap first filter only (trivially bypassed by rephrasing). Real defence: rules in system prompt, retrieved text clearly delimited as data, model with strong instruction hierarchy, optional classifier, and **least privilege** — the bot can't do damage even if fooled.
Indirect injection (malicious text inside an indexed doc)	—	The bigger RAG-specific risk. Only index trusted sources, sanitize on ingestion, never let retrieved text trigger tool calls / actions without checks.
Data leakage across users	—	Tenant filter on every retrieval, derived from auth — never from request body.
PII	Mask emails	Mask emails, phones, card numbers, national IDs (Presidio or similar); mask in logs too; re-check output.
Abuse / cost attacks	Rate limit per IP	Rate limit per API key/user (IPs are shared/rotated), max input length, token budgets, spend alerts.

9.3 Hardened main.py

The notes' version is a good skeleton. Gaps fixed below: no authentication; the cache is an unbounded dict (memory leak, and not shared between workers); only emails masked; the agent is simulated; no citations or fallback.

```python
import hashlib, re, time
from contextlib import asynccontextmanager
from cachetools import TTLCache                      # uv add cachetools  (use Redis for multi-worker)
from fastapi import FastAPI, Request, HTTPException, Depends, Header
from pydantic import BaseModel, Field
from pydantic_settings import BaseSettings
from slowapi import Limiter, _rate_limit_exceeded_handler
from slowapi.errors import RateLimitExceeded

class Settings(BaseSettings):                        # reads env vars / .env
    api_keys: set[str] = set()                       # API_KEYS='["k1","k2"]'
    cache_ttl_s: int = 3600
settings = Settings()

class ChatRequest(BaseModel):
    message: str = Field(..., min_length=1, max_length=4000)
    thread_id: str = Field(default="default", max_length=64)

class Citation(BaseModel):
    source: str
    page: int | str | None = None

class ChatResponse(BaseModel):
    response: str
    citations: list[Citation]
    thread_id: str
    cached: bool
    processing_time_ms: float

# ---------- security ----------
INJECTION = [re.compile(p, re.I) for p in (
    r"ignore\s+(all\s+)?(previous|prior|above)\s+instructions",
    r"(reveal|print|show)\s+(your\s+)?(system\s+)?prompt",
    r"you\s+are\s+now\s+",
)]
PII = {
    "[EMAIL]": re.compile(r"[\w.+-]+@[\w-]+\.[\w.-]+"),
    "[PHONE]": re.compile(r"(?:\+91[\s-]?)?[6-9]\d{9}\b"),
    "[CARD]":  re.compile(r"\b(?:\d[ -]?){13,16}\b"),
}
def sanitize(text: str) -> str:
    if any(p.search(text) for p in INJECTION):
        raise HTTPException(400, "Request blocked by input policy.")
    for tag, pat in PII.items():
        text = pat.sub(tag, text)
    return text

def get_tenant(x_api_key: str = Header(...)) -> str:
    if x_api_key not in settings.api_keys:
        raise HTTPException(401, "Invalid API key")
    return hashlib.sha256(x_api_key.encode()).hexdigest()[:12]   # map key → tenant id

def key_func(request: Request) -> str:               # rate-limit per API key, not per IP
    return request.headers.get("x-api-key", request.client.host)

# ---------- app ----------
cache: TTLCache = TTLCache(maxsize=10_000, ttl=settings.cache_ttl_s)
limiter = Limiter(key_func=key_func)

@asynccontextmanager
async def lifespan(app: FastAPI):
    app.state.graph = build_rag_graph()              # load vector store, models once at startup
```

```python
    yield

app = FastAPI(title="Production RAG API", version="1.2.0", lifespan=lifespan)
app.state.limiter = limiter
app.add_exception_handler(RateLimitExceeded, _rate_limit_exceeded_handler)

@app.post("/chat", response_model=ChatResponse)
@limiter.limit("20/minute")
async def chat(request: Request, payload: ChatRequest, tenant: str = Depends(get_tenant)):
    t0 = time.perf_counter()
    msg = sanitize(payload.message)
    key = hashlib.sha256(f"{tenant}:{' '.join(msg.lower().split())}".encode()).hexdigest()

    if (hit := cache.get(key)) is not None:
        return ChatResponse(**hit, thread_id=payload.thread_id, cached=True,
                            processing_time_ms=(time.perf_counter() - t0) * 1000)

    # RAGState from Ch 6, extended with `tenant: str` used as the retrieval metadata filter
    state = await request.app.state.graph.ainvoke({
        "question": msg, "query": msg, "tenant": tenant, "documents": [],
        "relevance": 0.0, "retry_count": 0, "max_retries": 2, "generation": ""})
    out = {
        "response": state["generation"],
        "citations": [Citation(source=d.metadata.get("source", "?"), page=d.metadata.get("page"))
                      for d in state["documents"]],
    }
    cache[key] = out
    return ChatResponse(**out, thread_id=payload.thread_id, cached=False,
                        processing_time_ms=(time.perf_counter() - t0) * 1000)

@app.get("/health")
def health():
    return {"status": "healthy", "version": app.version}
```

LLM FALLBACK IN ONE LINE

Inside `build_rag_graph()`, create the model as `llm = primary.with_retry(stop_after_attempt=2).with_fallbacks([secondary])` — if the primary provider errors or times out, LangChain transparently calls the secondary.

CACHE + CHAT HISTORY

Caching by question text is only safe for **stateless** Q&A. If answers depend on earlier messages in the thread, include a hash of the relevant history in the key, or don't cache follow-up turns. And clear/version the cache when the index is updated, or users get stale answers.

9.4 Dockerfile (non-root)

DOCKERFILE

```dockerfile
FROM python:3.12-slim

ENV PYTHONUNBUFFERED=1 PYTHONDONTWRITEBYTECODE=1 PIP_NO_CACHE_DIR=1
WORKDIR /app

RUN apt-get update && apt-get install -y --no-install-recommends curl \
    && rm -rf /var/lib/apt/lists/*

# deps first -> this layer stays cached until requirements.txt changes
COPY requirements.txt .
RUN pip install -r requirements.txt
```

```
RUN useradd --create-home appuser
COPY --chown=appuser:appuser . .
USER appuser

EXPOSE 8000
HEALTHCHECK --interval=30s --timeout=5s --retries=3 \
  CMD curl -fsS http://localhost:8000/health || exit 1

CMD ["uvicorn", "main:app", "--host", "0.0.0.0", "--port", "8000", "--workers", "2"]
```

SEEDHI BAAT — HAR LINE KYUN

slim image = chhota size, kam vulnerabilities. **requirements pehle copy** = code badalne pe dependencies dobara install nahi hoti (fast builds). **non-root user** = container hack ho bhi jaye toh root access nahi. **HEALTHCHECK** = orchestrator ko pata chalta hai app zinda hai ya restart karna hai. **--workers 2** = 2 processes; isi wajah se in-memory cache/rate-limit workers ke beech share nahi hota → Redis.

9.5 Render blueprint (render.yaml)

YAML

```yaml
services:
  - type: web
    name: production-rag-api
    runtime: docker          # build from the Dockerfile above (or: runtime: python)
    region: oregon
    plan: starter            # free plan sleeps when idle -> 30-60 s cold starts
    healthCheckPath: /health
    autoDeploy: true
    envVars:
      - key: OPENAI_API_KEY
        sync: false          # set secret in dashboard, not in git
      - key: LANGSMITH_TRACING
        value: "true"
      - key: API_KEYS
        sync: false
```

DEPLOYMENT GOTCHAS

- **Ephemeral disk:** on most PaaS (Render, Railway, Cloud Run) the container filesystem is wiped on redeploy — a local `./chroma_db` disappears. Use a persistent disk or, better, a managed vector DB / Postgres with pgvector.
- **Free-tier sleep:** free instances spin down after inactivity; the first request then takes ~a minute. Fine for a portfolio demo, not for production.
- **Region:** place the API near your vector DB and LLM endpoint; cross-region hops add latency on every request.

10 Interview Q&A

Original notes ke 5 sawaal + 10 aur jo SDE / GenAI interviews mein aksar aate hain. Answers 30–60 second mein bolne layak.

Q1. Why does fixed-size chunking cause retrieval failures?
It cuts at arbitrary character counts, splitting sentences, code blocks and definitions. Each half is an incomplete thought, so its embedding matches neither the full question nor the full answer well. Recursive/structure-aware splitting with overlap keeps units of meaning intact.

Q2. Define Reciprocal Rank Fusion. Why use it for hybrid search?
RRF(d) = Σ 1/(k + rank_m(d)) over retrievers, k ≈ 60. BM25 and cosine scores live on different, incomparable scales; RRF uses only ranks, so no normalization is needed, and documents ranked well by both retrievers rise to the top. k damps the advantage of rank 1 so no single retriever dominates.

Q3. How is Contextual Retrieval different from standard chunking?
Before indexing, an LLM writes a 1–2 sentence context for each chunk using the full document (company, period, section) and prepends it; the enriched text goes into both embeddings and BM25. In Anthropic's tests, failures dropped 35% (embeddings only), 49% (with contextual BM25), 67% (plus reranking). Prompt caching keeps the indexing cost low.

Q4. Trade-offs of HNSW M and ef_search?
M (links per node) raises recall but increases memory and build time. ef_search (candidate list at query time) raises recall but increases latency. ef_construction affects build quality/time only. Tune by measuring recall@k against exact search vs p95 latency.

Q5. When would you choose multimodal RAG (ColPali)?
When meaning lives in layout: tables, charts, scanned forms, slides, drawings. Text extraction scrambles those. Cost: many vectors per page, a multi-vector store, and a vision LLM for generation.

Q6. RAG vs fine-tuning — when each?
RAG for knowledge that is large, private, changing, or needs citations/access control. Fine-tuning for behaviour: format, tone, domain style, classification. They're complementary; facts injected by fine-tuning are hard to update and can't be cited.

Q7. A user says "the bot gave a wrong answer". How do you debug?
Open the trace. (1) Was the right chunk retrieved at all? If not → retrieval problem (chunking, query mismatch, filters, k). (2) Retrieved but ranked low/cut by budget → reranking/budget. (3) In context but answer wrong → generation problem (prompt, model, faithfulness). Add the case to the eval set so it can't regress.

Q8. What is a bi-encoder vs a cross-encoder?
Bi-encoder embeds query and document independently — vectors precomputed, so search over millions is fast, but less accurate. Cross-encoder scores the (query, doc) pair jointly — accurate but must run per pair, so only used to rerank a small candidate set (top 20–50).

Q9. How do you evaluate a RAG system?
Separate retrieval and generation. Retrieval: hit@k, MRR, context precision/recall on a labelled set. Generation: faithfulness (claims supported by context) and answer relevancy, typically with an LLM judge calibrated against human labels. Run in CI on every change; monitor refusal rate, thumbs-down and latency in production.

Q10. Why must the same embedding model be used for indexing and querying?
Each model defines its own vector space; vectors from different models aren't comparable. Switching models means re-embedding the corpus — so store raw chunk text and version the index (blue-green re-index, then switch).

Q11. How do you handle multi-tenant data isolation?
Store tenant_id as metadata on every chunk; derive tenant from the auth token server-side; apply it as a mandatory filter on every query (or separate collections/partitions per tenant for strong isolation). Never trust a tenant id sent in the request body. Scope caches per tenant too.

Q12. How do you keep the index fresh when documents change?
Deterministic chunk IDs (hash of source + position), track source → chunk ids, on change delete old chunks for that source and upsert new ones; run incrementally via a queue or scheduled job; invalidate caches for affected answers. LangChain's indexing API (RecordManager) automates the bookkeeping.

Q13. What is "lost in the middle" and how do you handle it?
LLMs use information at the start and end of long contexts more reliably than the middle. Handle it by sending fewer, better chunks (rerank + budget) and placing the most relevant chunks first (or at both ends).

Q14. What is HyDE?
Hypothetical Document Embeddings: the LLM drafts a plausible answer to the query, and you embed that draft instead of the raw question. Answers resemble documents more than questions do, improving recall for short or vague queries. Costs one extra LLM call; the draft itself is never shown to the user.

Q15. How would you reduce cost of a RAG API at scale?
Caching (exact, then carefully-thresholded semantic), smaller model routing for easy questions, token budgets and fewer/better chunks via reranking, prompt caching for static system prompts, reduced embedding dims + quantization for storage, and batching embeddings during ingestion. Measure $/request per stage first — the LLM call usually dominates.

PROJECT TIE-IN
When you describe a RAG project on your resume, name the concrete choices and numbers: "recursive 400-token chunks, hybrid BM25 + vector with RRF, cross-encoder rerank to top-4; hit@5 went from X% to Y% on a 60-question eval set; p95 latency Z ms". Interviewers remember measured improvements far more than a list of libraries.

A Corrections to the Original Notes

Notes kaafi ache the — ye woh jagah hain jahan cheezein galat, outdated ya incomplete thi.

#	Notes said	Accurate version
1	`uv add … fast-api`	Package name is `fastapi`.
2	Contextual Retrieval "reduces failures by 49% alone"	35% with contextual embeddings alone; 49% with contextual embeddings + contextual BM25; 67% adding reranking.
3	Chroma result printed as "Score"	`similarity_search_with_score` returns a distance (lower = better). Chroma defaults to L2 unless `hnsw:space` is set to cosine at creation.
4	Dim reduction 1536→512 saves "30–60%"	Saves 66.7% of vector storage.
5	Float32→Int8 saves "50–75%"	Exactly 75%. (50% is float16.)
6	Token estimate = words × 1.3	Rough for English prose only; use a tokenizer (tiktoken) for enforced budgets — code, JSON and Hindi differ a lot.
7	LangGraph code	Referenced `generate_response` / `fallback_response` nodes that were never added, no END edges, never compiled. Fixed in 6.2. Also keep original question separate from rewritten query.
8	API key hard-coded in Python; `datetime.utcnow()`	Keys via env/secrets; `datetime.now(timezone.utc)` (utcnow deprecated in 3.12).
9	`LANGCHAIN_TRACING_V2` etc.	Current names are `LANGSMITH_TRACING`, `LANGSMITH_API_KEY`, `LANGSMITH_PROJECT` (old names still accepted).
10	`langchain_community.vectorstores.Chroma`, `langchain.retrievers.*`	Use `langchain_chroma.Chroma`; in LangChain ≥1.0 legacy retrievers moved to `langchain_classic`.
11	Gemini embeddings = 768 dims	768 for text-embedding-004; gemini-embedding-001 defaults to 3,072 (truncatable).
12	"High-dimension embeddings" fix embedding mismatch	Main fixes are query rewriting / multi-query / HyDE and hybrid BM25.
13	"~90% of naive RAG systems fail"	Popular blog claim, not a measured statistic.
14	In-memory dict cache, per-IP rate limit, regex-only injection defence	Unbounded memory, not shared across workers, easily bypassed. Use TTL/Redis cache, per-key limits, layered injection defence + least privilege + tenant filters.
15	Cost table numbers	Treat as rough, dated estimates; verify against current vendor pricing.
16	render.yaml `env: python`, `plan: free`	`runtime:` is the current key; free plan sleeps (cold starts) and disk is ephemeral — a local Chroma DB is lost on redeploy.

B One-Page Cheat Sheet

Revision ke liye — interview se pehle bas ye page.

DEFAULT PRODUCTION RECIPE

1. Clean text, rich metadata, deterministic chunk IDs
2. Recursive split, ~300–500 tokens, 10–15% overlap
3. (Optional) contextual prefix per chunk
4. text-embedding-3-small (cosine), HNSW M=16
5. Hybrid: vector top-20 + BM25 top-20 → RRF (k=60)
6. Cross-encoder rerank → top 3–5
7. Token budget; best chunks first
8. Grounded prompt + citations + refusal string
9. Low relevance → rewrite (≤2) → fallback
10. Trace everything; eval set in CI

NUMBERS TO REMEMBER

RRF k = 60 · overlap 10–20% · pgvector defaults M=16, ef_construction=64, ef_search=40 · 1M × 1536 float32 ≈ 6.1 GB · int8 = −75% · 1536→512 = −66.7% · Contextual Retrieval −35/−49/−67% · ~4 chars per English token

SYMPTOM → FIX

Right doc never retrieved → query rewrite, HyDE, hybrid, check filters
Codes/IDs not found → BM25 hybrid
Retrieved but buried → reranker
Answer missing context → bigger chunks / parent-child / contextual retrieval
Made-up answer → grounding prompt, threshold, faithfulness check
Slow → trace stages; cache; smaller model; fewer chunks
Expensive → cache, routing, budgets, prompt caching
Other tenant's data → mandatory server-side tenant filter

FORMULAS

$\cos(u,v) = u \cdot v / (\|u\|\|v\|)$
$\|u-v\|^2 = 2 - 2\cdot\cos(u,v)$ (unit vectors)
$\text{RRF}(d) = \Sigma\ 1/(k + \text{rank})$
$\text{BM25} = \Sigma\ \text{IDF} \cdot f(k_1+1) / (f + k_1(1-b+b\cdot|d|/\text{avgdl}))$
$\text{Bytes} = N \times d \times \text{bytes_per_dim}$

— End —